

# Security Review — Inbox

COTI PoD Inbox contracts · InboxBase inheritance chain · 12-agent parallel audit

| FIELD                | VALUE                                                                                    |
|----------------------|------------------------------------------------------------------------------------------|
| Mode                 | Named files (4)                                                                          |
| Files reviewed       | InboxBase.sol (820) · InboxEstimateGas.sol (142) · InboxMiner.sol (421) · Inbox.sol (43) |
| Repository           | coti-pod-inbox-contracts @ 89121ad (main)                                                |
| Method               | 12 parallel adversarial agents (Claude Opus), deduplicated, four-gate validation         |
| Confidence threshold | 80 — at or above gets a fix; below gets description only                                 |
| Findings             | 12 findings · 9 leads                                                                    |

**Evidence status — please read.** No finding in this report has an executable proof-of-concept. The evidence supporting each is static: traced call paths, EVM gas and storage-slot arithmetic, and quoted source. Finding 4 additionally had its shipped configuration values verified directly against scripts/deploy-utils.ts. Confidence scores reflect how completely the attack path was traced, not execution against a live EVM. Findings should be reproduced in a test harness before remediation work is scheduled or a severity is assigned externally.

## Summary

| #  | CONF. | TITLE                                                              | LOCATION                                       | AGENTS |
|----|-------|--------------------------------------------------------------------|------------------------------------------------|--------|
| 1  | 90    | Post-call gas reserve smaller than the failure write it guarantees | InboxMiner._runIncomingExecution               | 10     |
| 2  | 90    | Return-leg payload never checked against prepaid budget            | InboxBase._reply                               | 7      |
| 3  | 85    | Callback fee floor priced from request size, not reply size        | InboxBase.sendTwoWayMessage                    | 1      |
| 4  | 85    | Shipped fee template makes admission band unreachable              | InboxBase._createRequest                       | 2      |
| 5  | 80    | Estimate suppresses LOG opcodes charged to the metered subcall     | InboxEstimateGas._estimateExecutionGasForMiner | 1      |
| 6  | 80    | Expiry clock runs while pause disables the cure                    | InboxMiner.retryFailedRequest                  | 1      |
| 7  | 80    | Permissionless retry executes target with caller-chosen gas        | InboxMiner.retryFailedRequest                  | 1      |
| 8  | 75    | Miner-reject hatch re-applies the cap that blocked the item        | InboxMiner._ingestMinerReject                  | 2      |
| 9  | 75    | System-error return leg is fail-unsafe                             | InboxBase._sendSystemErrorCallbackWithCode     | 3      |
| 10 | 75    | Ingest-storage fee returned to sender as executable gas            | InboxMiner._runIncomingExecution               | 1      |

|    |    |                                                                      |                                               |   |
|----|----|----------------------------------------------------------------------|-----------------------------------------------|---|
| 11 | 75 | TTL terminalization half-writes the record, emits nothing on one-way | <code>InboxMiner.retryFailedRequest</code>    | 3 |
| 12 | 72 | Ordinary execution failure produces no error return leg              | <code>InboxMiner._runIncomingExecution</code> | 1 |

**Fix these two together.** Findings 1 and 4 interact. Clamping the fee band (4) pins every COTI-bound `targetFee` at exactly 25,000,000, which is precisely the condition that makes the gas-reserve shortfall (1) bind on every single ingest rather than occasionally.

## Findings

### 1. Post-call gas reserve is smaller than the failure write it guarantees

90

InboxMiner.\_runIncomingExecution · 10 of 12 agents · gas-reserve-undersized

#### DESCRIPTION

POST\_CALL\_GAS\_RESERVE = 200\_000 is documented as ensuring "failure accounting can always commit", but errors[rid] = Error{bytes32, uint64, bytes(256)} is 11 cold zero→nonzero SSTOREs = **243,100 gas** before two LOGs — so a target that burns its allowance and reverts with a full 256-byte payload OOGs the whole batchProcessRequests, and nonce contiguity forbids skipping it. ESTIMATE\_OUTER\_RESERVE = 150\_000 has the same defect and is worse: the errors[rid] write is *not* gated by \_shouldEmit(), so the identical 243,100 gas is spent during an estimate that always reverts anyway.

#### IN PLAIN ENGLISH

The inbox is a delivery depot that always sets aside a little fuel so it can fill out an incident report when a delivery goes wrong. The amount set aside is less than the report actually costs to file. So if a package fails messily enough, the depot runs out of fuel halfway through the paperwork — and because the paperwork can't be finished, the *entire truckload* is rejected, not just the one bad package. Since packages must be processed in strict numbered order, nobody can skip past the bad one.

#### IMPACT IF NOT FIXED

Anyone can deploy a contract on the destination chain and, for the price of a single message fee, force miners to lose the full gas of an entire batch transaction — repeatedly. Because messages must be processed in unbroken numerical order, the offending message cannot be skipped, so the lane between the two chains **stops entirely** until a miner manually intervenes with the reject mechanism (which finding 8 shows can itself fail). Every pending deposit, withdrawal and transfer on that lane freezes. This does not require malice to trigger: an ordinary Error(string) with a 200-character message already exceeds the threshold, so it will also happen by accident in production.

#### FIX — OPTION A (RAISE THE RESERVE)

```
- uint256 private constant POST_CALL_GAS_RESERVE = 200_000;
- uint256 private constant ESTIMATE_OUTER_RESERVE = 150_000;
+ uint256 private constant POST_CALL_GAS_RESERVE = 320_000;
+ uint256 private constant ESTIMATE_OUTER_RESERVE = 300_000;
```

#### FIX — OPTION B (SHRINK WHAT IS PERSISTED)

```
- errors[rid] = Error({requestId: rid, errorCode: ERROR_CODE_EXECUTION_FAILED, errorMessage:
returnData});
+ errors[rid] = Error({requestId: rid, errorCode: ERROR_CODE_EXECUTION_FAILED,
+ errorMessage: abi.encodePacked(keccak256(returnData))});
```

#### FIX — APPLY INDEPENDENTLY (ESTIMATE MUST NOT WRITE AT ALL)

```
- errors[rid] = Error({requestId: rid, errorCode: ERROR_CODE_EXECUTION_FAILED, errorMessage:
returnData});
- if (_shouldEmit()) { emit ErrorReceived(rid, ERROR_CODE_EXECUTION_FAILED, returnData); }
+ if (_shouldEmit()) {
+   errors[rid] = Error({requestId: rid, errorCode: ERROR_CODE_EXECUTION_FAILED, errorMessage:
returnData});
+   emit ErrorReceived(rid, ERROR_CODE_EXECUTION_FAILED, returnData);
+ }
```

## 2. Return-leg payload size is never checked against the prepaid callback budget

90

InboxBase.\_reply · 7 of 12 agents · unpriced-return-leg

### DESCRIPTION

`_requireReplyMethodCallBounded` bounds the reply only by the admin constant `maxReplyMethodCallBytes` (default 8192) and `_createRequest` only by `maxMethodCallBytes` — neither is a function of `callerFee`. A destination target can answer a minimum-fee 288-byte request with an 8 KB reply the source-chain miner must ingest at roughly 8–13× the funded cost, repeatably. Proof the caps are mutually incoherent: with the shipped Sepolia template, `expectedMinFee(8192) = 10,287,600` exceeds `maxExecutionGas = 5,000,000`, so a maximum-size reply is unfundable at *any* legal `callerFee`.

### IN PLAIN ENGLISH

You pay postage based on the size of the letter you send. The reply you get back can be thirty times bigger, and nobody charges anyone for that — the courier absorbs it. Worse still: the largest reply the rules allow costs more to deliver than the largest postage anyone is permitted to pay. So a maximum-size reply literally cannot be paid for, no matter how much the sender wants to spend.

### IMPACT IF NOT FIXED

A direct, repeatable economic drain on whoever runs the miners — in practice, the protocol operator. An attacker controlling both endpoints pays roughly one eighth of the real cost and pockets the difference as damage. This is a slow bleed rather than an outage: nothing breaks immediately, but running a miner becomes unprofitable, and when miners stop relaying, the bridge stops. Because the shortfall is structural rather than configuration-dependent (when `constantFee > 0`, reply size is priced at exactly zero), no operator tuning closes it.

### FIX — OPTION A (BOUND THE REPLY BY WHAT WAS BOUGHT)

```
function _requireReplyMethodCallBounded(MpcMethodCall memory methodCall) internal view {
    uint256 weight = MinerRejectLib.structuralSize(methodCall);
    if (weight > maxReplyMethodCallBytes()) revert ResponseOutOfBounds(weight,
maxReplyMethodCallBytes());
+    uint256 required = _expectedMinFeeGasUnits(weight, _remoteMinFeeConfigMem());
+    if (required > incomingRequest.callerFee) revert ResponseOutOfBounds(required,
incomingRequest.callerFee);
}
```

### FIX — OPTION B (CHARGE WORST-CASE REPLY AT SEND TIME)

```
- if (callerGasLocalUnits < expectedMinFee(dataSize, localMin)) revert
CallbackFeeTooLow(callerGasLocalUnits);
+ if (callerGasLocalUnits < expectedMinFee(maxReplyMethodCallBytes(), localMin)) revert
CallbackFeeTooLow(callerGasLocalUnits);
```

### 3. Callback fee floor is priced from the request's size, not the reply's

85

InboxBase.sendTwoWayMessage · wrong-size-input-to-fee-formula

#### DESCRIPTION

One `dataSize = abi.encode(methodCall).length` is computed from the outbound request and passed to `_validateAndPrepareTwoWayFees` for **both** the target-leg and callback-leg minimums, while the protocol's own quoter `calculateTwoWayFeeRequiredInLocalToken` takes `remoteMethodCallSize` and `callbackMethodCallSize` as separate parameters — so the validator's floor and the quoter's price disagree whenever the reply differs in size from the request. This is finding 2 seen at its cause rather than its symptom.

#### IN PLAIN ENGLISH

The price list has two columns — "size of your letter" and "size of the reply" — but the cashier only ever reads the first column and uses that number for both. The quote you were shown and the amount the till actually demands are computed differently.

#### IMPACT IF NOT FIXED

Wallets and front-ends display the quoted fee before a user signs. When the request is large and the reply small, the on-chain requirement exceeds the quote and the transaction **reverts after the user has already committed and paid gas**. When the reply is large, the user under-pays and the shortfall lands on the miner. Either way the displayed fee is unreliable, which is corrosive to user trust in a bridge where a failed transaction can look indistinguishable from lost funds.

#### FIX

```
- (uint256 targetFeeGas, uint256 callerFeeGas) = _validateAndPrepareTwoWayFees(dataSize, msg.value,
callbackFeeLocalWei);
+ (uint256 targetFeeGas, uint256 callerFeeGas) =
+   _validateAndPrepareTwoWayFees(dataSize, maxResponseSize, msg.value, callbackFeeLocalWei);

// add maxResponseSize as a caller-declared parameter bounded by maxReplyMethodCallBytes(),
// store it on the Request, and enforce it in _requireReplyMethodCallBounded
```

## 4. Shipped fee template makes the admission band one gas unit wide and unreachable

85

InboxBase.\_createRequest · 2 agents + independently verified · degenerate-fee-band

### DESCRIPTION

FeeManager.expectedMinFee returns constantFee verbatim in constant-fee mode, so the floor and \_createRequest's maxExecutionGas ceiling are **both 25,000,000** in the shipped FEE\_CONFIG\_COTI\_SIDE — a band of width 1 — while reachable values are quantized at  $\text{localPrice}/\text{remotePrice} \times \text{gasPriceMul}/\text{Div}$  ( $\approx 60,000$  at realistic ETH/COTI prices). 25,000,000 is not a multiple of 60,000, so no msg.value satisfies both bounds. Config confirmed directly at deploy-utils.ts:392–403 and its installation as remote at line 450. CI misses it because every test sets both prices to  $1e18$ , collapsing the step to 1. Note the file's own comment at line 390 reads "floor is lower", suggesting the author believed headroom existed.

### IN PLAIN ENGLISH

A shop where the minimum price is exactly £25 and the maximum price is also exactly £25 — but the till only accepts one denomination of coin, worth £1.50. There is no number of coins that makes exactly £25. Pay one coin less and you're under the minimum; one coin more and you're over the maximum. Nobody can ever complete a purchase. The shop's own test till was set up with 1p coins, so it always worked in rehearsal.

### IMPACT IF NOT FIXED

Operationally the most severe finding in this report. If the shipped configuration is deployed as-is against realistic token prices, **every single message from Sepolia or Avalanche to COTI reverts** — not a degradation but a total outage of the entire bridge, present from the moment of deployment and affecting every user simultaneously. No deposit, withdrawal or transfer can be initiated. The failure is silent in testing and would first surface on mainnet. It is also fragile in the other direction: even if a particular price ratio happened to divide evenly, any subsequent price movement would break it again.

### FIX — OPTION A (CLAMP INSTEAD OF REVERT)

```
- if (targetFeeGas > remoteMax.maxExecutionGas) revert FeeGasTooHigh(targetFeeGas,  
remoteMax.maxExecutionGas);  
+ if (targetFeeGas > remoteMax.maxExecutionGas) targetFeeGas = remoteMax.maxExecutionGas;
```

### FIX — OPTION B (REJECT A DEGENERATE CONFIG AT VALIDATION)

```
function _requireValidFeeConfig(LibFeeStorage.FeeConfig memory feeConfig) private pure {  
+   if (feeConfig.constantFee > 0 && feeConfig.maxExecutionGas == feeConfig.constantFee) {  
+       revert FeeConfigInvalid(feeConfig);  
+   }
```

## 5. Estimate suppresses LOG opcodes that are charged to the subcall it is metering

80

InboxEstimateGas.\_estimateExecutionGasForMiner · estimate-underreports-gas

### DESCRIPTION

`emit ResponseReceived` and `emit MessageSent` (plus the keccak in `_methodCallLogData`) execute *inside* the metered target subcall on a respond path, but are skipped under `_shouldEmit()`. `gasUsed` is measured strictly around that subcall, so it under-reports by ~8 gas per response byte — unbounded in reply size — and any `targetFee` sized from it under-funds the real execution.

### IN PLAIN ENGLISH

The dress rehearsal skips the scenes with the fireworks, then reports how long the show takes. The real performance includes the fireworks and runs longer than the rehearsal said it would. Anyone who booked the theatre based on the rehearsal timing books too short a slot.

### IMPACT IF NOT FIXED

Miners size their batch transactions from this estimate. A systematic under-report means transactions that run out of gas — which then triggers finding 1 and reverts the entire batch rather than one message. This is the mechanism that makes finding 1 reachable in ordinary operation rather than only under deliberate attack, so the two compound: honest traffic produces the under-estimate, and the under-estimate produces the batch failure.

### FIX

```
- if (_shouldEmit()) { emit ResponseReceived(incomingRequestId, data); }
+ if (_shouldEmit()) { emit ResponseReceived(incomingRequestId, data); }
+ else { _estimateSuppressedLogGas += 375 * 3 + 8 * data.length; }

// add _estimateSuppressedLogGas to the gasUsed reported by ExecutionGasEstimate
```

## 6. Expiry clock runs while the pause switch disables the cure

80

InboxMiner.retryFailedRequest · ttl-clock-runs-while-paused

### DESCRIPTION

`messageProcessingPaused` reverts `retryFailedRequest` at its first line while `block.timestamp` keeps advancing against the ingest-stamped `incomingRequest.timestamp`. Any pause longer than `maxMessageLife` (48 h default) converts every pending execution-failed request into a permanently dead one that any unprivileged party can finalize the instant the pause lifts. `setMaxMessageLife(1)` reproduces the same retroactive sweep in a single transaction.

### IN PLAIN ENGLISH

A parcel is held for 48 hours before being written off as lost. The depot closes for a week during an emergency. When it reopens, every parcel is now more than 48 hours old and gets written off at once — even though the depot being closed is precisely why nobody could collect them. The first person through the door on reopening day can trigger the mass write-off.

### IMPACT IF NOT FIXED

The emergency pause — the exact tool an operator reaches for during an incident — becomes the thing that destroys every in-flight recoverable message the moment it is lifted. Users' prepaid fees are consumed producing failure notices instead of the operations they paid for, and the messages become permanently unrecoverable. The safety mechanism turns into the damage mechanism at the worst possible moment, and the resulting loss is attributable to the operator's own incident response rather than to the original incident.

### FIX

```
+ uint256 private _lastUnpausedAt;
+ function setMessageProcessingPaused(bool paused) external onlyOwner {
+     if (!paused && messageProcessingPaused) _lastUnpausedAt = block.timestamp;
+     messageProcessingPaused = paused;

- if (life != 0 && block.timestamp > uint256(incomingRequest.timestamp) + uint256(life)) {
+ uint256 start = Math.max(uint256(incomingRequest.timestamp), _lastUnpausedAt);
+ if (life != 0 && block.timestamp > start + uint256(life)) {
```



## 7. Permissionless retry executes the target with caller-chosen gas

80

InboxMiner.retryFailedRequest · caller-controlled-gas-steering

### DESCRIPTION

The retry branch sets `gasForCall = gasleft()` and deliberately ignores the prepaid budget, so an attacker choosing a low transaction gas limit can steer a gas-sensitive target into its degraded branch — the target's own nested call runs out of gas under EIP-150's 63/64 rule while the outer frame survives and returns cleanly. On that non-reverting return, `delete errors[requestId]` makes the outcome permanent and all later retries revert `RetryFailedRequestNotAFailedRequest`.

### IN PLAIN ENGLISH

Anyone may re-attempt a failed delivery, and whoever attempts it chooses how much time to allow. A hostile person allows only just enough — so the recipient half-completes the job and signs for it anyway. The record now says "delivered successfully", and no further attempts are permitted, ever.

### IMPACT IF NOT FIXED

An attacker can force a partial or degraded outcome on somebody else's cross-chain operation and permanently lock it in, at the cost of one transaction. The victim application has no second chance and no on-chain signal distinguishing this from a genuine recovery — `RetryFailedRequestSuccess` is emitted either way. For a token bridge this is the difference between "your withdrawal is pending" and "your withdrawal completed with the wrong amount and cannot be retried".

### FIX

```
- if (kind == IncomingExecKind.Retry) { gasForCall = gasleft(); }
+ if (kind == IncomingExecKind.Retry) {
+   gasForCall = _localRequestExecutionBudget(incomingRequest.targetFee);
+   require(gasleft() > gasForCall + POST_CALL_GAS_RESERVE, "retry underfunded");
+ }
```

## 8. Miner-reject escape hatch re-applies the cap that made the item unrejectable

75

InboxMiner.\_ingestMinerReject · 2 agents · escape-hatch-revert · below threshold, description only

### DESCRIPTION

The reject branch skips the three admission caps so it can always clear a stuck nonce, but `_sendSystemErrorCallbackWithCode` → `_createRequest` re-checks the same stored `callerFee` against the same `remoteMinFeeConfig.maxExecutionGas`. After a cap reduction, both the normal branch and the escape hatch revert on the identical value and the lane wedges. The two-way reject path is untested — `test/InboxSizeCaps.ts` builds its only reject from a one-way send, so this code never executes in CI.

### IN PLAIN ENGLISH

The emergency lever that clears a jammed conveyor belt is itself wired through the jam. Pulling it does nothing, because the thing blocking the belt also blocks the lever.

### IMPACT IF NOT FIXED

The single recovery mechanism for an unprocessable message fails in exactly the circumstance it exists for. Combined with strict message ordering, a routine fee-config change wedges a chain-to-chain lane permanently. The only remaining escape is for a miner to submit deliberately falsified data (claiming the message was one-way), which silently discards the sender's error callback and loses their prepaid fee with no record. Requires an admin config change to trigger, which is why it sits below the fix threshold — but config changes are routine operations, not exotic ones.

## 9. System-error return leg is fail-unsafe

75

InboxBase.\_sendSystemErrorCallbackWithCode · 3 agents · below threshold, description only

### DESCRIPTION

The helper already handles `callerFee == 0` by emitting locally and returning, but has no equivalent guard for an over-cap `callerFee` or an over-size error payload — so `_createRequest` can revert inside it. That takes down the whole batch on the encode-failure and miner-reject paths, and permanently bricks `retryFailedRequest`'s TTL branch, where the request becomes neither retryable nor terminalizable.

### IN PLAIN ENGLISH

The "we're sorry, your delivery failed" notice can itself fail to send. And when it does, it doesn't just get skipped — it brings down the entire depot with it.

### IMPACT IF NOT FIXED

Converts isolated per-message failures into batch-wide failures across three separate call paths. On the expiry path there is no workaround at all: the message can neither be retried nor declared dead, and the prepaid fee is stranded permanently with no mechanism to recover or refund it. The existing `callerFee == 0` early-return shows the fail-safe pattern was understood — it was simply not extended to the other two ways this call can fail.

## 10. Ingest-storage fee is returned to the sender as executable gas

75

InboxMiner.\_runIncomingExecution · fee-accounting-double-spend · below threshold, description only

### DESCRIPTION

`expectedMinFee` charges `dataSize * gasPerByte` explicitly to price destination ingest storage (`MEASURED_INGEST_GAS_PER_BYTE = 800`, commented "measured ingest ~690+"), but `_localRequestExecutionBudget` subtracts only the `errorLength` term. That same allotment is handed back as the subcall gas cap while the miner has already spent it on SSTOREs. *Scope note: the defective expression is in `FeeManager.localRequestExecutionBudget`, outside the four reviewed files; the in-scope consumer is `_runIncomingExecution`.*

### IN PLAIN ENGLISH

You're charged separately for storing your parcel and for handling it. The depot then gives the storage money back to you as extra handling time — while still paying the storage cost out of its own pocket. It collects for one thing and delivers two.

### IMPACT IF NOT FIXED

Structural under-collection of roughly 36% per message, growing linearly with payload size. Unlike finding 2, this affects **ordinary honest traffic**, not just attacker-crafted messages — every message the bridge carries loses the operator money. The end state is the same as finding 2 (miners become unprofitable and stop), but it arrives without anyone attacking the system, simply as a function of normal usage volume.

## 11. TTL terminalization writes half the record and emits nothing on one-way

75

InboxMiner.retryFailedRequest · 3 agents · below threshold, description only

### DESCRIPTION

The TTL branch writes only `errors[requestId].errorCode = ERROR_CODE_EXPIRED`, leaving `errorMessage` holding the prior execution revert data — so `getOutboxError` returns code 4 paired with stale bytes. And because `_sendSystemErrorCallbackWithCode` returns early when `errorSelector == 0`, which `sendOneWayMessage` enforces for every one-way request, permanent terminalization of a one-way message emits **zero events**.

### IN PLAIN ENGLISH

When a parcel is written off, only the status stamp is updated — the "reason" field still contains the previous, different explanation. And for one whole category of parcel, the write-off is announced to nobody at all. It simply becomes true, silently.

### IMPACT IF NOT FIXED

Monitoring, indexing and support tooling cannot distinguish a still-recoverable message from a permanently dead one, and will decode the wrong failure reason when reading the code/message pair that the interface documents as belonging together. Operations teams lose visibility into precisely the failures they most need to see, and users contacting support about a stuck transfer cannot be given an accurate answer. No funds move incorrectly — the harm is entirely to observability — which is why it sits below the fix threshold.

## 12. Ordinary execution failure produces no error return leg

72

`InboxMiner._runIncomingExecution` · below threshold, description only

### DESCRIPTION

The `if (!success)` branch records the error but never calls `_sendSystemErrorCallback`, even though the sender was compelled to register a non-zero `errorSelector` and was charged an `errorLength * gasPerByte` reserve for exactly that purpose. The only producer is the voluntary, unrewarded TTL branch of `retryFailedRequest` — which is unreachable entirely when `maxMessageLife == 0`, a documented and supported setting.

### IN PLAIN ENGLISH

If the recipient formally refuses your parcel, you're told. If the depot's own machinery jams, you're told. But if the recipient simply crashes when opening it, nobody tells you — you just wait. After 48 hours a passing volunteer might file the paperwork, if one happens along. If the depot has switched off the 48-hour rule, nobody ever files it.

### IMPACT IF NOT FIXED

Applications waiting on a two-way message hang with user funds escrowed, dependent on an unrewarded volunteer calling a public function that nobody is obliged to call. Under the default 48-hour setting this resolves slowly and badly; with `maxMessageLife = 0` it never resolves at all. For the privacy portal specifically, this is the difference between a deposit that eventually becomes refundable and one that sits in limbo indefinitely, with the user's collateral locked and no on-chain path to release it.

## Leads

---

Vulnerability trails with concrete code smells where the full exploit path could not be completed in one analysis pass. These are not false positives — they are high-signal leads for manual review. Not scored.

- **Unchecked length overflow in assembly decoder** — `InboxBase._tryDecodeAbiBytes.iszero(gt(add(0x40, len), retLen))` uses unchecked `add`, so `len ≥ 2256 - 0x40` wraps and passes, yielding a `bytes` whose length word flows into `_callWithCappedReturnData` as the CALL input size. Unreachable today only because `mpcAbiReEncode` is fixed at `init` with no setter. Flagged by 6 of 12 agents; one-word fix.
- **Gas reserve silently skipped** — `InboxMiner._computeUserCallGas.if (gasForCall > outerReserve) { gasForCall -= outerReserve; }` has no `else`, so when `gasLeft() ≤ outerReserve` the reserve is abandoned entirely and the target receives 63/64 of everything.
- **Permissionless estimate as forged-context primitive** — `InboxEstimateGas._estimateExecutionGasForMiner`. No `onlyMiner`, no `nonReentrant`, no pause check, no duplicate guard; overwrites live `incomingRequests[]` and calls an arbitrary target with forged `inboxMsgSender()`. Contained *only* by the unconditional terminal revert.
- **Uncapped gas on the re-encode delegatecall** — `InboxBase._safeEncodeMethodCall`. Forwards all gas with no reserve, before any budget applies; the non-reverting recovery costs ~1.1M gas while EIP-150 leaves only 1/64.
- **Unpriced ingest storage at protocol ceiling** — `InboxMiner.batchProcessRequests`. A 32,768-byte payload costs 22.6M gas of cold SSTOREs inside the contiguity-critical loop. Demoted: the shipped `gasPerByte = 800` prices this correctly and reaching the ceiling needs admin config beyond defaults.
- **Cross-chain cap mirroring is unenforced** — `InboxBase._createRequest` / `InboxMiner.batchProcessRequests`. Lane liveness depends on three inequalities between two independently-owned templates that cannot be updated atomically; ingest reverts rather than skipping.
- **Front-runnable initializer** — `Inbox.init`. No caller check, `_disableInitializers()` deliberately omitted, both DELEGATECALL targets set once with no setters. Mitigation is entirely off-chain.
- **System error masquerades as response** — `InboxBase.getInboxResponse.inboxResponses[]` carries both genuine target responses and Inbox-generated error payloads with no discriminator.
- **Read-then-refresh oracle ordering** — `InboxBase.sendTwoWayMessage` / `sendOneWayMessage`. Fee computed from the cache, `refreshCache()` called after; the sender always pays the pre-refresh ratio and subsidizes the next caller's update.

**Out of scope, surfaced incidentally.** `MpcExecutorCotiProxyInbox.registerExecutor` has a completely unguarded one-shot setter (first-caller-wins) and sits under `contracts/mpc/coti-side/`, not the test tree.

`FeeManager.setMaxReplyMethodCallBytes` rejects only zero, with no protocol ceiling unlike `maxMethodCallBytes`.

⚠ This review was performed by an AI assistant. AI analysis can never verify the complete absence of vulnerabilities and no guarantee of security is given. No finding in this report has an executable proof-of-concept; evidence is static analysis, gas arithmetic and traced call paths. Team security reviews, bug bounty programs, and on-chain monitoring are strongly recommended. For a consultation regarding your projects' security, visit <https://www.pashov.com>